

Proceso de software ← conjunto estructurado de actividades, métodos y prácticas para desarrollar y mantener software de manera sistemática y predecible.

- Especificación de requisitos
- Análisis
- Diseño
- Implementación
- Pruebas
- Despliegues
- Mantenimiento

Modelos de proceso de software

- Reducción de riesgos
- Comunicación efectiva
- Mejora continua
- Transferencia de conocimiento
- Predictibilidad y estimación

• Modelo en cascada

- Enfoque secuencial
- Variantes:
 - Clásico
 - Con retroalimentación
 - Con prototipado

• Modelo de desarrollo incremental

- Enfoque incremental
- Modelos:
 - Modelo general
 - Modelo en espiral ← iterativo y orientado a riesgos
 - Modelo RUP (Rational Unified Process) ← por cosas de uso
 - Metodologías Ágiles ← integración de lo existente

• Modelo basado en reutilización ← Integración de lo existente

• Modelo orientado a la implantación (DevOps) ← prioriza entrega continua y despliegue eficiente.

Scrum → Metodología Ágil

↑ Conjunto de roles, eventos, artefactos y reglas.

roles del equipo {

- Scrum Máster
- Dueño del Producto → gestión de la pila del producto
- Desarrolladores → crear la pila del Sprint.
Adaptar la planificación para lograr el objetivo del Sprint.



Autogestionado

Multidisciplinar

Sin jerarquías

Responsabilidad Colectiva

Sprint → iteración de duración fija

- Se define la pila del Sprint (trabajo a realizar).
- Scrum diario → progreso hacia el objetivo del Sprint.
- Revisión del Sprint → resultado del Sprint.
- Retrospectiva del Sprint → formas de aumentar la calidad y efectividad.

Análisis = Modelado Conceptual

Modelado Estructural

Diagrama de clases ← estructura estática de un sistema (clases y relaciones)

Clases ← requisitos y caso de uso.

Clase asociación ← relación asociación con propiedades.

Relaciones

Diagrama de objetos

Modelado del Comportamiento

Diagrama de secuencia ← ordenación temporal de los mensajes.

Actores

Objetos

Líneas de vida ← $\vdots$ <<create>> <<destroy>>

Activación ← $\square$ objetos realizando una una operación

Mensajes

→ síncrono, espera respuesta

--> asíncrono, continúa con la ejecución

Bucles y condiciones

alt ← if/else

opt ← if

loop

Diagrama transición de estados

Casos de uso extendidos ← secuencia concreta y detallada de pasos que describe una ejecución específica de un caso de uso.

Tipos de escenario:

- Éxito

- Fracaso (alternativo)

Elementos:

Nombre

Actores (involucrados)

Precondiciones

Flujo principal

Flujos alternativos

Postcondiciones

Requisitos funcionales $\leftarrow$ Qué hace el sistema.

Funciones, servicios y comportamientos específicos.

Requisitos NO funcionales $\leftarrow$ Cómo se comporta el sistema.
Restricciones y cualidades.

Ingeniería de requisitos

Definición de requisitos

Extracción

Análisis

Especificación

Validación

Gestión de requisitos (Reqmine)

Modelado de requisitos (UML)

Diagramas Estructurales $\leftarrow$ Estáticas

- Clases, Objetos, Componentes, Despliegue, Paquetes.

Diagramas de Comportamiento $\leftarrow$ Dinámicas

- Casos de Uso, Interacción (Secuencia, Comunicación), Estados, Actividades.

Casos de uso $\leftarrow$ funcionalidad completa que proporciona valor al actor.

Actores $\leftarrow$ representan un rol que interactúa con el sistema.

Relaciones

- Asociación: conexión básica entre un actor y un caso de uso. Indica que el actor participa en esa funcionalidad.

- Include: un caso de uso base siempre incorpora la funcionalidad de otro caso de uso incluido. Es obligatorio.

- Extend: un caso de uso puede añadir comportamiento adicional a otro de forma opcional.

- Herencia: un caso de uso o actor hijo hereda las asociaciones del actor/caso de uso padre.

